

MoneyPoly White-Box Testing Report

Total white-box test cases: 35

Passed: 35

Failed: 0

Pylint score: 10.00/10

1. White-Box Scope And Verified State

MoneyPoly was audited at the control-flow and state-transition level. The white-box suite covers setup validation, board routing, card handling, special-tile resolution, purchase and rent flows, mortgages, trades, jail logic, bankruptcy, net-worth calculation, winner selection, and turn-order edge cases.

The suite was re-run locally and all 35 collected cases passed. Static analysis was also re-run on the whitebox code and the audited files scored 10.00/10 under pylint.

2. Pylint Iterations

The Git history shows six dedicated pylint-cleanup iterations, which keeps style and design cleanup separate from logic fixes:

- Iteration 1: Remove unused imports and replace broad exception handling
- Iteration 2: Simplify boolean checks and tidy control-flow style issues
- Iteration 3: Add missing module, function, and class docstrings
- Iteration 4: Wrap long card definitions and normalize formatting
- Iteration 5: Refactor card handling and resolve remaining pylint design warnings

These iterations cover import cleanup, exception narrowing, boolean simplification, documentation, formatting, and remaining design warnings.

3. Error Fix History

- Error 1: Validate game setup for minimum player count and unique names
- Error 2: Fix dice to use the full six-sided range
- Error 3: Award salary when passing or landing on Go
- Error 4: Require full ownership for group rent bonus
- Error 5: Reduce bank reserves when issuing loans and ignore negative collections
- Error 6: Handle empty card decks safely in remaining-count and repr methods
- Error 7: Allow property purchase at exact balance
- Error 8: Reject purchase of owned or mortgaged properties
- Error 9: Credit property owners when rent is paid
- Error 10: Preserve mortgage state when unmortgage payment fails

- Error 11: Prevent partial mortgage completion when the bank lacks funds
- Error 12: Transfer trade cash to the seller
- Error 13: Reject invalid trade requests such as negative cash and self-trade
- Error 14: Deduct the jail fine from the player on voluntary release
- Error 15: Reject auctions for owned or mortgaged properties
- Error 16: Add railroad property objects so railroad tiles are resolvable
- Error 17: Call the interactive menu before rolling each normal turn
- Error 18: Include owned properties in player net-worth calculation
- Error 19: Select the highest net-worth player as winner
- Error 20: Fix turn order after bankruptcy so the next player is not skipped
- Error 21: Cancel doubles extra-turn logic if the player was eliminated

4. Complete White-Box Case Catalog

Case W-01:

whitebox/tests/test_moneypoly_whitebox.py::test_board_tile_type_and_purchasable_branches

Scenario: Board tile type and purchasable branches.

Why Needed: This case checks the board routing helpers that decide what kind of tile a player has landed on and whether a purchase branch should open at all.

Expected Result: Go, property, chest, and blank tiles should be classified correctly; purchasable should be true only for unowned and unmortgaged properties.

Observed Result: Passed locally. This case protects the ownership and purchasable-state branches used by movement resolution.

Case W-02:

whitebox/tests/test_moneypoly_whitebox.py::test_board_ownership_helpers_track_owned_and_unowned_properties

Scenario: Board ownership helpers track owned and unowned properties.

Why Needed: This case verifies that helper methods reporting owned and unowned properties stay

aligned with the true board state.

Expected Result: Properties owned by a player should appear in the owned list and should not appear in

the unowned list.

Observed Result: Passed locally. It guards later features such as trades, bankruptcy cleanup, and winner

calculations.

Case W-03:

whitebox/tests/test_moneypoly_whitebox.py::test_card_deck_cycles_and_handles_empty_deck

Scenario: Card deck cycles and handles empty deck.

Why Needed: The card deck is a stateful structure, so white-box coverage must include both the normal

cycling path and the empty-deck safety path.

Expected Result: A non-empty deck should cycle in order, while an empty deck should return None

safely and report zero remaining cards.

Observed Result: Passed locally. This case exposed the earlier empty-deck crash path fixed in Error 6.

Case W-04:

whitebox/tests/test_moneypoly_whitebox.py::test_game_requires_at_least_two_unique_players

Scenario: Game requires at least two unique players.

Why Needed: Game setup is a gatekeeper branch; invalid player lists must be rejected before the main

loop begins.

Expected Result: A one-player game and a game with duplicate names should both raise ValueError.

Observed Result: Passed locally. This case maps to Error 1.

Case W-05: whitebox/tests/test_moneypoly_whitebox.py::test_dice_roll_uses_full_six_sided_range

Scenario: Dice roll uses full six sided range.

Why Needed: Movement, doubles logic, and jail behavior depend on the dice using the correct inclusive range.

Expected Result: The dice should call randint with bounds 1 and 6 for both dice and should recognize doubles correctly.

Observed Result: Passed locally. This case maps to Error 2.

Case W-06:

whitebox/tests/test_moneypoly_whitebox.py::test_player_move_collects_salary_when_passing_go

Scenario: Player move collects salary when passing go.

Why Needed: Passing Go changes both position and balance, so the move branch needs explicit state validation.

Expected Result: Crossing the board boundary should wrap the position and award Go salary.

Observed Result: Passed locally. This case maps to Error 3.

Case W-07:

whitebox/tests/test_moneypoly_whitebox.py::test_property_group_requires_full_set_for_bonus_rent

Scenario: Property group requires full set for bonus rent.

Why Needed: Rent depends on ownership state across multiple properties, so partial and full-group

ownership must be separated clearly.

Expected Result: Bonus rent should apply only when one owner controls the full set; partial ownership

should keep base rent.

Observed Result: Passed locally. This case maps to Error 4.

Case W-08: whitebox/tests/test_moneypoly_whitebox.py::test_bank_give_loan_reduces_bank_reserves

Scenario: Bank give loan reduces bank reserves.

Why Needed: Bank reserve state is a core invariant; giving a loan should never create money out of thin

air.

Expected Result: Giving a loan should increase the player balance and reduce the bank balance by the

same amount.

Observed Result: Passed locally. This case maps to part of Error 5.

Case W-09: whitebox/tests/test_moneypoly_whitebox.py::test_bank_collect_and_pay_out_guards

Scenario: Bank collect and pay out guards.

Why Needed: Bank helper methods need defensive paths for zero, negative, and overdraw situations.

Expected Result: Negative collection should be ignored, zero payout should be harmless, and an

oversized payout should raise ValueError.

Observed Result: Passed locally. This case maps to part of Error 5.

Case W-10:

whitebox/tests/test_moneypoly_whitebox.py::test_buy_property_allows_spending_entire_balance

Scenario: Buy property allows spending entire balance.

Why Needed: Exact-boundary money states are a classic white-box edge case and often reveal off-by-one affordability bugs.

Expected Result: A player with balance exactly equal to the price should be allowed to buy and should end at balance zero.

Observed Result: Passed locally. This case maps to Error 7.

Case W-11:

whitebox/tests/test_moneypoly_whitebox.py::test_buy_property_rejects_owned_or_mortgaged_property

Scenario: Buy property rejects owned or mortgaged property.

Why Needed: Purchase routing must reject invalid property states before mutating ownership or money.

Expected Result: Buying an owned or mortgaged property should return False and leave ownership unchanged.

Observed Result: Passed locally. This case maps to Error 8.

Case W-12: whitebox/tests/test_moneypoly_whitebox.py::test_pay_rent_transfers_money_to_owner

Scenario: Pay rent transfers money to owner.

Why Needed: Rent is a two-sided transaction, so both debit and credit paths need direct coverage.

Expected Result: The tenant balance should fall by the rent and the owner balance should rise by the same amount.

Observed Result: Passed locally. This case maps to Error 9.

Case W-13:

whitebox/tests/test_moneypoly_whitebox.py::test_unmortgage_property_stays_mortgaged_when_player_cannot_afford

Scenario: Unmortgage property stays mortgaged when player cannot afford.

Why Needed: Failed-payment branches must leave property state unchanged; otherwise the game can drift into impossible states.

Expected Result: Unmortgaging should fail when the player cannot afford it and the property should remain mortgaged.

Observed Result: Passed locally. This case maps to Error 10.

Case W-14:

whitebox/tests/test_moneypoly_whitebox.py::test_trade_transfers_cash_to_seller_and_property_to_buyer

Scenario: Trade transfers cash to seller and property to buyer.

Why Needed: Trading changes ownership lists and both balances at once, so the whole state transition needs end-to-end validation.

Expected Result: The property should move to the buyer and cash should move to the seller in the same operation.

Observed Result: Passed locally. This case maps to Error 12.

Case W-15:

whitebox/tests/test_moneypoly_whitebox.py::test_trade_rejects_negative_cash_and_self_trade

Scenario: Trade rejects negative cash and self trade.

Why Needed: Trade validation must reject malformed or self-referential requests before any mutation

occurs.

Expected Result: Negative-cash trades and self-trades should return False.

Observed Result: Passed locally. This case maps to Error 13.

Case W-16:

whitebox/tests/test_moneypoly_whitebox.py::test_handle_jail_turn_deducts_player_when_paying_fine

Scenario: Handle jail turn deducts player when paying fine.

Why Needed: The jail-release path combines confirmation input, bank transfer, and movement, so it

needs direct branch coverage.

Expected Result: Choosing to pay the fine should release the player, deduct the fine, credit the bank, and

continue movement.

Observed Result: Passed locally. This case maps to Error 14.

Case W-17:

whitebox/tests/test_moneypoly_whitebox.py::test_handle_property_tile_routes_buy_auction_skip_and_rent

Scenario: Handle property tile routes buy auction skip and rent.

Why Needed: Property tile handling contains several decision branches, so one case has to walk the full

buy, auction, skip, and rent routing logic.

Expected Result: The handler should dispatch to the correct downstream action depending on ownership

and the player choice.

Observed Result: Passed locally. This case protects multiple purchase and rent routing paths.

Case W-18:

whitebox/tests/test_moneypoly_whitebox.py::test_mortgage_property_stays_consistent_when_bank_cannot

_pay

Scenario: Mortgage property stays consistent when bank cannot pay.

Why Needed: Mortgage operations also need rollback safety; a failed bank payout must not leave the

property half-mutated.

Expected Result: If the bank cannot pay, the mortgage call should fail and the property should remain in

the original state.

Observed Result: Passed locally. This case maps to Error 11.

Case W-19:

whitebox/tests/test_moneypoly_whitebox.py::test_move_and_resolve_special_tiles[go_to_jail-30-0]

Scenario: Special-tile branch for go to jail at board position 30.

Why Needed: This case covers the movement branch for the Go To Jail tile.

Expected Result: Landing on Go To Jail should send the player to jail immediately instead of treating the

tile like a normal landing.

Observed Result: Passed locally. This parameterized case covers the jail-routing branch inside

movement resolution.

Case W-20:

whitebox/tests/test_moneypoly_whitebox.py::test_move_and_resolve_special_tiles[income_tax-4-200]

Scenario: Special-tile branch for income tax at board position 4.

Why Needed: This case covers the income-tax branch where cash is deducted and transferred to the

bank.

Expected Result: Landing on Income Tax should deduct the configured income tax or the configured tax

amount correctly and keep movement state consistent.

Observed Result: Passed locally. This parameterized case covers the tax-payment branch inside

movement resolution.

Case W-21:

whitebox/tests/test_moneypoly_whitebox.py::test_move_and_resolve_special_tiles[luxury_tax-38-75]

Scenario: Special-tile branch for luxury tax at board position 38.

Why Needed: This case covers the luxury-tax branch where a fixed deduction is applied.

Expected Result: Landing on Luxury Tax should deduct the configured amount and keep the player on

the correct square.

Observed Result: Passed locally. This parameterized case covers the second tax branch inside

movement resolution.

Case W-22:

whitebox/tests/test_moneypoly_whitebox.py::test_move_and_resolve_special_tiles[free_parking-20-0]

Scenario: Special-tile branch for free parking at board position 20.

Why Needed: This case covers the free-parking branch where no monetary side effect should occur.

Expected Result: Landing on Free Parking should not trigger purchase, tax, or jail behavior.

Observed Result: Passed locally. This parameterized case covers the no-op special-tile branch.

Case W-23:

whitebox/tests/test_moneypoly_whitebox.py::test_move_and_resolve_draws_and_applies_cards

Scenario: Move and resolve draws and applies cards.

Why Needed: Landing on card spaces is a routing branch that joins board movement with deck handling.

Expected Result: The correct deck should be used and the drawn card should be forwarded to the card-

application logic.

Observed Result: Passed locally. This case protects card-draw dispatch.

Case W-24:

whitebox/tests/test_moneypoly_whitebox.py::test_move_and_resolve_routes_property_like_tiles_to_property

handler

Scenario: Move and resolve routes property like tiles to property handler.

Why Needed: Movement resolution must recognize property-like tiles and forward them into property

handling instead of treating them as blanks.

Expected Result: Property-like tiles reached through movement should route to the property handler.

Observed Result: Passed locally. This case supports the fix set around property-like movement routing.

Case W-25: whitebox/tests/test_moneypoly_whitebox.py::test_railroad_tiles_have_property_objects

Scenario: Railroad tiles have property objects.

Why Needed: Railroads must actually exist as property objects or the movement and rent paths for those

tiles break silently.

Expected Result: Railroad board positions should return Property objects.

Observed Result: Passed locally. This case maps to Error 16.

Case W-26: whitebox/tests/test_moneypoly_whitebox.py::test_apply_card_branches

Scenario: Apply card branches.

Why Needed: The card application function contains many internal branches, so direct white-box

coverage is required for each action family.

Expected Result: Collect, pay, jail, jail-free, move, birthday, and similar card actions should mutate state

correctly.

Observed Result: Passed locally. This case protects the internal decision tree of `_apply_card()`.

Case W-27:

whitebox/tests/test_moneypoly_whitebox.py::test_apply_card_collect_from_all_skips_players_without_enough_money

Scenario: Apply card collect from all skips players without enough money.

Why Needed: Collect-from-all is a multi-player edge case where some players may not be able to contribute.

Expected Result: Players who cannot pay should be skipped safely while the acting player still receives valid contributions.

Observed Result: Passed locally. This case covers an affordability-sensitive card branch.

Case W-28:

whitebox/tests/test_moneypoly_whitebox.py::test_check_bankruptcy_removes_player_and_resets_properties

Scenario: Check bankruptcy removes player and resets properties.

Why Needed: Bankruptcy changes the active player list and the ownership map, so it is one of the most important destructive branches in the game.

Expected Result: A bankrupt player should be removed and their properties should be reset.

Observed Result: Passed locally. This case supports the later turn-order fixes.

Case W-29:

whitebox/tests/test_moneypoly_whitebox.py::test_player_net_worth_includes_owned_property_values

Scenario: Player net worth includes owned property values.

Why Needed: Winner selection depends on net-worth calculation, so the valuation helper needs direct coverage.

Expected Result: Owned property values should contribute to player net worth.

Observed Result: Passed locally. This case maps to Error 18.

Case W-30: whitebox/tests/test_moneypoly_whitebox.py::test_find_winner_returns_richest_player

Scenario: Find winner returns richest player.

Why Needed: The final game result is a business-critical branch and should pick the player with the highest net worth.

Expected Result: The winner helper should return the richest player.

Observed Result: Passed locally. This case maps to Error 19.

Case W-31:

whitebox/tests/test_moneypoly_whitebox.py::test_auction_property_rejects_owned_or_mortgaged_property

Scenario: Auction property rejects owned or mortgaged property.

Why Needed: Auctions should not proceed on invalid property states, or the game can violate ownership rules.

Expected Result: Owned or mortgaged properties should be rejected from the auction flow.

Observed Result: Passed locally. This case maps to Error 15.

Case W-32:

whitebox/tests/test_moneypoly_whitebox.py::test_safe_int_input_returns_default_on_invalid_entry

Scenario: Safe int input returns default on invalid entry.

Why Needed: The CLI must remain usable under invalid input instead of crashing or hanging.

Expected Result: Invalid numeric input should return the supplied default value.

Observed Result: Passed locally. This case protects CLI resilience.

Case W-33:

whitebox/tests/test_moneypoly_whitebox.py::test_play_turn_calls_interactive_menu_before_rolling

Scenario: Play turn calls interactive menu before rolling.

Why Needed: The normal turn flow should expose the interactive menu before a roll, otherwise that path

becomes dead code from the player point of view.

Expected Result: play_turn should invoke the menu before rolling the dice.

Observed Result: Passed locally. This case maps to Error 17.

Case W-34:

whitebox/tests/test_moneypoly_whitebox.py::test_play_turn_bankruptcy_does_not_skip_next_player

Scenario: Play turn bankruptcy does not skip next player.

Why Needed: Elimination during a turn can corrupt the turn pointer, so that edge case needs explicit

control-flow coverage.

Expected Result: When a player goes bankrupt, the next surviving player should still get the next turn.

Observed Result: Passed locally. This case maps to Error 20.

Case W-35:

whitebox/tests/test_moneypoly_whitebox.py::test_bankrupt_player_does_not_grant_extra_turn_after_doubl

es

Scenario: Bankrupt player does not grant extra turn after doubles.

Why Needed: Doubles logic and bankruptcy interact in a subtle way, making this an important white-box

edge case.

Expected Result: A player eliminated during a doubles turn should not receive an extra turn.

Observed Result: Passed locally. This case maps to Error 21.